

# 4th class–

## 1- How python works:

i- Compilation process:

While running a Python script (.py), the Python compiler first checks your code for syntax errors. If the code is valid, it translates the source code into an intermediate format called Bytecode.

i-What is Bytecode: It is a low-level, platform-independent set of instructions. It isn't machine code that CPU understands. Only Pvm only understand bycode instructions. it is specific to Python.

ii-It is often stored in a folder named `__pycache__` with a .pyc extension.

ii. Interpretation

The Bytecode is then sent to the Python Virtual Machine (PVM). The PVM contains the Interpreter,

i-The Interpreter reads the Bytecode line-by-line.

ii-It converts each line into Machine Code (binary 0s and 1s) that your specific computer hardware can execute.

## 2. Compiler

**What it does:** It translates the entire source code into machine code all at once before the program runs.

## 3. Interpreter

**What it does:** It translates and executes the code line-by-line during runtime.

# **–5th class–**

## **1. Data Types in Python**

**Data types 5 types such as:**

- Int: numbers (e.g., 10, -5).
  - Float: Decimal numbers (e.g., 10.5, 3.14).
  - String: in quotes (e.g., "Hello").
  - bool: either True or False.
  - None: It indicates that a variable exists but doesn't have a value yet.
- 

## **2. Operators in Python**

- + (Addition), - (Subtraction), \* (Multiplication), / (Division)
- // (Floor Division): Returns the whole number after division (e.g.,  $7 // 2 = 3$ ).
- % (Modulus): Returns the remainder (e.g.,  $7 \% 2 = 1$ ).
- (Exponent): Power (e.g.,  $2^3 = 8$ ).

### **Another Comparison Operators**

- == (Equal), != (Not Equal)
- > (Greater than), < (Less than)
- >= (Greater than or equal), <= (Less than or equal)

### **C. Logical Operators**

- and: Returns True if both statements are true.
- or: Returns True if one statement is true.
- not: Reverses the result (e.g., not (True) is False)

## **D. Assignment Operators**

Used to assign values to variables.

- = (Assignment), += (Add and assign), -= (Subtract and assign).

## ----6th class---

### 1. Input & Type Conversion

By default, the `input()` function always takes data as a String.

- Concatenation: Using `+` with inputs will join them as text (e.g., `"10" + "20"` results in `"1020"`).
- Type Casting: To perform math, need to convert the string to an integer using `int()`.
  - *Example:* `result = int(first) + int(second)`

### 2. String Formatting Approaches

several ways to combine strings and variables:

- f-strings : The most modern and readable way.
  - *Syntax:* `f"Hello {name}"`
- `.join()` Method: *Syntax:* `" ".join([first_name, last_name])`
- `.format()` Method: Uses placeholders `{}` to insert values.
  - *Syntax:* `"{} {}".format(a, b)`, if i extend 1 variable, then will be needed `{}` more extra one.
- String Multiplication: Make repetition a string multiple times using `*`.
  - *Example:* `"sorry " * 1000`

### 3. Useful String Methods

**Python built-in functions:**

- `.upper()` / `.lower()`: Converts text to all capital or all small letters.
- `.title()`: Capitalizes the first letter of every word.
- `.capitalize()`: Capitalizes only the very first letter of the string.
- `.swapcase()`: Flips cases (Lower to right , Upper to Lower).
- `.count("x")`: Counts how many times a specific character appears.
- `.find("word")`: Returns the index (position) of the first occurrence of a word.

#### **4. Variable Overriding**

In Python, variables are dynamic. If i assign a new value to an existing variable name, the old value is replaced (overridden).

- *Example:* If `number = 10` and then `number = 30`, the final value stored is 30.

## —7th Class—

### Python String Methods and Operations

## 1. Fundamental String Operations

Python strings are indexed sequences of characters where indexing starts at 0 and negative indexing starts from the end at -1.

| Operation              | Description                                           | Example (x = "hello world") |
|------------------------|-------------------------------------------------------|-----------------------------|
| len(x)                 | Returns the total number of characters in the string. | 11                          |
| Indexing [i]           | Accesses a character at a specific position.          | x[4] → 'o'                  |
| Negative Indexing [-i] | Accesses characters starting from the end.            | x[-1] → 'd'                 |

### String Slicing

**i**-x[0:5] / x[:5]: Extracts characters from index 0 up to (but not including) index 5 (example: "hello").

**ii**-x[6:]: Extracts characters from index 6 to the very end (example:"world").

## 2.String Modification Methods

1-.replace(): Replaces a specific part of the string with new text.

- `"rocky".replace("r", "p")` → output: `'pocky'`

2-.strip(): Removes extra spaces from both the start and the end.

- `" hello ".strip()` → output: `'hello'`

3.lstrip(): Removes spaces only from the left side.

- `" hello ".lstrip()` → output: `'hello '`

4.rstrip(): Removes spaces only from the right side.

- `" hello ".rstrip()` → output: `' hello'`

5.upper(): Converts all letters to uppercase.

- `"python".upper()` → output: `'PYTHON'`

6.lower(): Converts all letters to lowercase.

- `"PYTHON".lower()` → output: `'python'`

7.title(): Capitalizes the first letter of every word.

- `"hello world".title()` → output: `'Hello World'`

8.split(): Splits the string into a list based on a separator.

- `"a,b,c".split(",")` → output: `['a', 'b', 'c']`

9.capitalize(): Capitalizes only the first letter of the entire string.

- `"hello".capitalize()` → output: `'Hello'`

## Methods:

**1-.startswith():** Checks if the string starts with a specific value.

- `"apple".startswith("a")` → output: True

**2-.endswith():** Checks if the string ends with a specific value.

- `"apple".endswith("e")` → output: True

**3-.find():** Returns the index position of the first occurrence of a value.

- `"hello".find("e")` → output: 1

**4-.count():** Counts how many times a value appears in the string.

- `"banana".count("a")` → output: 3

**5-.isdigit():** Checks if all characters in the string are numbers.

- `"123".isdigit()` → output: True

**6-.isalpha():** Checks if all characters in the string are letters.

- `"hello".isalpha()` → output: True

## 4. String Validation

**Basically, return True or False.**

- `.isalpha()`: True if all characters are alphabetic (A-Z).
- `.isdigit()`: True if all characters are digits (0-9).



- `.isalnum()`: True if all characters are alphanumeric (letters or numbers).
- `.isspace()`: True if the string consists only of whitespace.
- `.istitle()`: True if the string follows title case rules.
- `.islower()`: True if all characters are lowercase.

## 5. Input and Output Formatting

### User Input

```
1-name = input("Enter your name: ")
```

```
2-age = int(input("Enter your age: "))
```

### Printing Techniques

1. Standard Print: `print("Name:", name, "Age:", age)` or,
2. F-String: `print(f"Your name is {name} and your age is {age}")`.

## —8th Class—

**if Statement:** Executes the code block only if the condition is **True**.

- `if age >= 18:` → **Result:** Checks if age is 18 or more.

**elif (else if):** Used to check multiple conditions one after another if the previous ones were False.

- `elif score >= 80:` → **Result:** Checks this only if the first `if` failed.

**else:** Executes only if none of the above conditions are met.

- `else :` → **Result:** The final option and if any of the condition fail then else condition will work

## —9th class—

The Python ternary operator evaluates a condition and returns a specific value in a single line

**Ternary operator** = [Value\_If\_True] if [Condition] else [Value\_If\_False].

```
status = "Pass" if marks >= 40 else "Fail"
```

i-**Logic:** If marks are 40 or more, status is "Pass"; otherwise, it is "Fail".

ii-**Output:** "Pass" (if marks = 50)